УРОК 16

AI для OSINT 2026

Інтеграція AI у робочі процеси

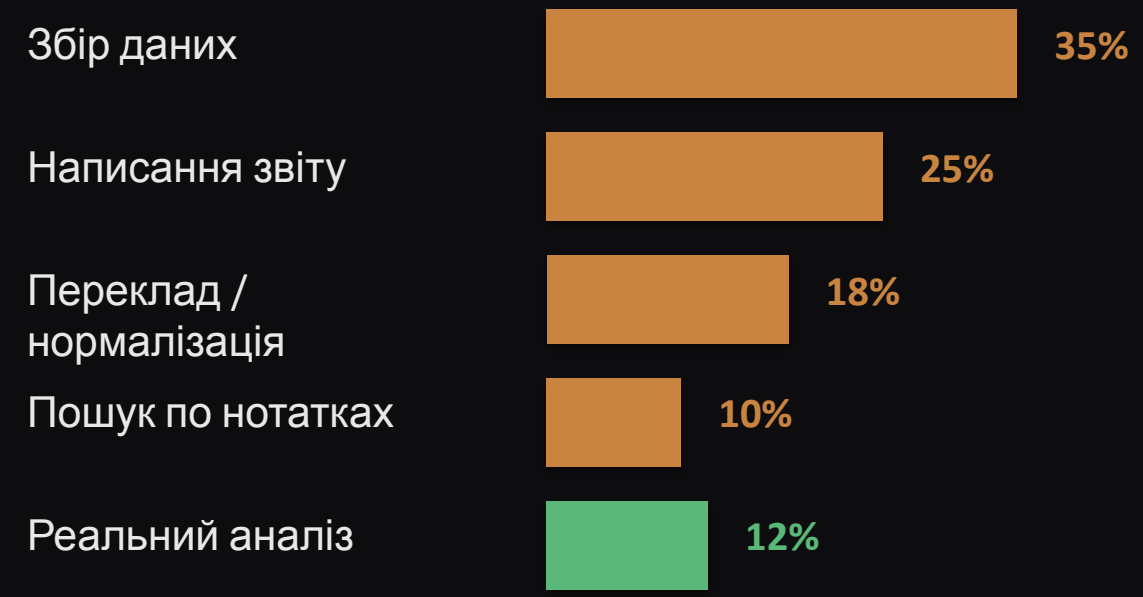
Звіти · Шаблони · Візуалізації

«AI не замінює аналітика — він забирає все, що не вимагає думати, щоб аналітик думав більше.»

Інструменти: Claude Code · Obsidian · Python · Plotly · Jinja2

КОНТЕКСТ

Де аналітик втрачає час



Лише 12% часу — на власне аналіз

AI бере на себе:

→ Переклад і перефразовування
→ Чернетки звітів
→ Генерація коду
→ Автоматичні графіки
→ Пошук по знаннях (RAG)

Аналітик залишає собі:

✗ Оцінка достовірності джерела
✗ Атрибуція операції
✗ Ключові судження з наслідками
✗ Судження під невизначеністю

План заняття

1	Розвідувальний звіт: BLUF, сім секцій, рівні впевненості, методологія	
2	Prompt engineering: архітектура промпту, R.O.C.E., ACH	▸ **ДЕМО 1 · Spot Report з AI**
3	Obsidian: шаблони, YAML frontmatter, Dataview	▸ **ДЕМО 2 · живий vault**
4	Python: pandas → Plotly → Jinja2; карта у звіті	
5	Агентні воркфлоу: ReAct, Claude Code, MCP	▸ **ДЕМО 3 · конвеєр і агент**

Кейси для демо: A — Rosoboronexport, реальні дані Заняття 9 · B — мережа Telegram-каналів, навчальний датасет.

120 хвилин · перерва після демо 1 · усі промпти з демо — у матеріалах заняття.

Планування перед написанням

«Справжня цінність — не в тому, що ви знайшли, а в тому, як ви це інтерпретували.»

1. Яка мета продукту?

Flash report · Assessment · Strategic estimate · Forensic report

2. Хто аудиторія?

Технічний фахівець · Керівник · Юрист · Медіа / громадськість

3. Яке рішення буде прийнято?

Якщо відповідь незрозуміла — уточніть у замовника ДО початку роботи

💡 *Промпт: «Я пишу OSINT-звіт для [аудиторія] з метою [ціль]. Яка структура найбільш підходить?»*

Анатомія розвідувального звіту

1	**ТИТУЛЬНА СТОРІНКА** Назва · Організація · Дата · Гриф · Номер кейсу
2	**BLUF** Bottom Line Up Front — ключовий висновок + рівень впевненості (1–3 речення)
3	**ВСТУП** Цілі · Обсяг · Дослідницькі питання · Значущість
4	**МЕТОДОЛОГІЯ ★** Інструменти · Джерела · Процес верифікації · Обмеження
5	**ЗНАХІДКИ** Хронологічно або тематично — факти відокремлені від оцінок
6	**АНАЛІЗ** Key Judgements + рівні впевненості · Альтернативи · Прогалини
7	**ВИСНОВКИ / РЕКОМЕНДАЦІЇ** Конкретні · Дієві · Пріоритизовані

BLUF — Bottom Line Up Front

Найважливіший висновок — ПЕРШОЮ фразою. Завжди.

[Ключовий висновок] + [Рівень впевненості] + [Найважливіший факт-обґрунтування]

✗ ПОГАНО

«Ми проаналізували активність 47 каналів... серед них виявлено повторюваний контент... час публікацій корелює... з урахуванням вищезазначеного, вважаємо за можливе припустити наявність координації.»

✓ ДОБРЕ (BLUF)

«З ВИСОКОЮ ВПЕВНЕНІСТЮ оцінюємо, що 10 із 47 досліджуваних Telegram-каналів діють як координована мережа — публікують ідентичний контент у вікні ±5 хвилин із синхронізацією учасницької бази 78–91%.»

💡 *Після написання чернетки → Claude: «Перепиши перший абзац у форматі BLUF: ключове судження першою фразою, рівень впевненості явно вказаний.»*

⚠ **Якщо не можеш написати BLUF — аналіз не завершений.**

Рівні впевненості та мова оцінок

ВИСОКА — **«з високою впевненістю оцінюємо»**

Численні докази · кілька незалежних джерел · несуперечлива картина

ПОМІРНА — **«оцінюємо» / «вважаємо ймовірним»**

Деякі докази · є прогалини або альтернативні пояснення

НИЗЬКА — **«допускаємо» / «не можемо виключити»**

Мало даних · одне джерело · значна невизначеність

Майже напевно	З великою ймовірн.	Ймовірно	≈ рівна ймовірн.	Малоймовірно	Дуже малоймовірно	Майже виключено
> 95%	75–95%	55–75%	45–55%	25–45%	5–25%	< 5%

⚠ **Кожне ключове судження ПОВИННО мати явний рівень впевненості.**

Методологія та оцінка джерел

Навіщо секція «Методологія»

→ Читач розуміє, звідки взялись висновки
→ Забезпечує відтворюваність аналізу
→ Юридичний захист — ланцюжок доказів
→ Виявляє власні прогалини явно

Що включати:

Інструменти — перелік + пояснення використання
Джерела — типи, часовий діапазон збору
Процес верифікації — як підтверджувались факти
Обмеження — що НЕ могло бути досліджено

★ ПРАВИЛО ТРЬОХ ДЖЕРЕЛ

Ключовий факт вважається встановленим лише якщо підтверджений трьома незалежними джерелами різного типу (первинне + вторинне + верифікуюче).

Стандарти цитування:

📅 Завжди вказуйте дату доступу — онлайн-контент змінюється і видаляється
💾 Зберігайте архівну копію (archive.org, локальний скріншот з timestamp)
🔗 Посилання на конкретне твердження, а не загальний сайт
🤖 AI-generated контент — НЕ є джерелом. Це інструмент синтезу.

AI у написанні звіту: що і як делегувати

Чернетка за нотатками	→ *«Напиши чернетку spot report у форматі BLUF + KJ на основі цих нотаток»*
Переклад	→ *«Переклади цей текст українською, збережи технічну термінологію»*
Нормалізація стилю	→ *«Перепиши в нейтральному аналітичному стилі без емоційних оцінок»*
Список альтернатив	→ *«Перелічи 5 альтернативних пояснень для цього патерну поведінки»*
Gap analysis	→ *«Які питання залишаються без відповіді після цього аналізу?»*
Executive summary	→ *«Напиши резюме у 3 реченнях для нетехнічної аудиторії»*

Практичний патерн:

1. Аналітик збирає факти → 2. AI структурує чернетку → 3. Аналітик додає атрибуції → 4. AI полірує стиль → 5. Аналітик підписує

Архітектура ефективного промпту

Промпт для аналітичної роботи — це не запитання. Це технічне завдання.

[РОЛЬ]	Ти — досвідчений OSINT-аналітик зі спеціалізацією в...
[КОНТЕКСТ]	Ми розслідуємо... Наразі встановлено, що...
[ЗАВДАННЯ]	Твоє завдання — ...
[ФОРМАТ]	Відповідь у форматі: 1) ... 2) ... 3) ...
[ОБМЕЖЕННЯ]	Не роби припущень без доказів. Якщо даних недостатньо — скажи явно.
[ВХІДНІ ДАНІ]	<текст / дані для аналізу>

⚠ Без ролі → загальна відповідь. Без формату → непридатна для звіту. Без обмежень → галюцинує «зручні» факти.

Фреймворк R.O.C.E.

R

ROLE

Роль

Ти — досвідчений аналітик відкритих джерел зі знанням цифрових слідів і атрибуції акаунтів.

O

OBJECTIVE

Ціль

Проведи аналіз цифрового сліду підозрюваного акаунту і визнач ймовірне місце базування оператора.

C

CONTEXT

Контекст

Акаунт @example_ua існує з 2019 р., активний о 9–18:00 UTC+2. Публікує проросійський контент. Раніше пов'язаний з @old_handle.

E

EVIDENCE

Докази

[вставити останні 30 повідомлень у форматі timestamp + текст]

Результат: мова оригіналу · ймовірний часовий пояс · тематичні патерни · зв'язок між акаунтами + рівні впевненості

Готові промпти для аналітичних задач

СИНТЕЗ ДЖЕРЕЛ

Нижче — 5 статей з різних джерел. Синтезуй:
1) Факти-консенсус 2) Суперечності 3) Унікальна інформація 4) Хронологія
Не додавай власних оцінок. Лише синтез.

ХРОНОЛОГІЯ

Побудуй хронологію у форматі:
YYYY-MM-DD HH:MM | Подія | Джерело | Достовірність (підтверджено/вірогідно/непідтверджено)
Де час невідомий — [приблизно]. Виділи розриви.

ВИЯВЛЕННЯ МАНІПУЛЯЦІЙ

Визнач: 1) Які факти включені/виключені? 2) Емоційне забарвлення? 3) Ключові слова-фрейми 4) Ознаки синхронізованого поширення
Для кожного пункту: цитата + рівень впевненості.

GAP ANALYSIS

Ось що нам відомо: [перелік]
Визнач: які ключові питання без відповіді?
Для кожної прогалини: чому важливо + яке джерело може заповнити.

★ Техніка «Why?»: після кожного висновку запитайте ще раз «Чому?» — це розкриває глибинне розуміння.

Analysis of Competing Hypotheses (ACH) з AI

Мета: ВІДКИДАТИ гіпотези через суперечності — не підтверджувати через збіги.

Докази	H1: Координована операція	H2: Органічний рух	H3: Комерційний накрут
Синхронізація ±5 хв	++	−−	+
Overlap учасників 78–91%	++	−	++
Ідентичний текст дослівно	++	−−	−
Різноманітні IP авторів	−	+	−
Монетизації немає	+	++	−−

✓ **Висновок: H1 має найменшу кількість суперечностей → найімовірніша гіпотеза.**

Демо 1 · Spot Report з AI

Що показую

1. Промпт за R.O.C.E.: роль, ціль, читач і його рішення, докази з файлу → чернетка Spot Report

2. Переписати перший абзац у BLUF: судження, рівень впевненості, обґрунтування; порівняти «було» і «стало»

3. Перевірка рівнів впевненості: таблиця «судження · рівень · скільки незалежних джерел»

4. ACH-матриця в Markdown: три гіпотези, докази з файлу, що відкидаємо і чому

5. Аналітик редагує: атрибуція, прогалини, підпис

На що дивитись

→ Промпт — це технічне завдання: без читача і його рішення виходить загальний текст

→ Кожне судження прив'язане до розділу досьє або номера факту

→ Кілька записів одного агрегатора — це одне джерело, а не підтвердження

→ ACH відкидає гіпотези через суперечності, а не шукає збіги

→ AI дає чернетку за хвилину; атрибуцію і підпис лишає людині

Кейс A · досьє Rosoboronexport: cases/roe/dossier.md

Кейс B · нотатки про мережу каналів: cases/channels/notes.md

Чернетка, BLUF і матриця — за п'ять хвилин. Час, що звільнився, іде на перевірку джерел і судження.

РИЗИКИ ТА ОБМЕЖЕННЯ

Де AI помиляється в аналітичній роботі

⚠ ГАЛЮЦИНАЦІЇ ФАКТІВ

LLM вигадує цитату, дату, ім'я або URL, якого не існує.
→ Правило: жоден факт у звіті без верифікації незалежним джерелом.
→ Промпт: «Якщо ти не впевнений — скажи 'невідомо', не вигадуй»

⚠ CONFIRMATION BIAS ЧЕРЕЗ ФОРМУЛЮВАННЯ

«Доведи, що це операція впливу» → знайде докази.
«Аргументи ПРОТИ операції впливу» → знайде інші.
→ Завжди запитуй обидві сторони. Використовуй ACH.

⚠ ПЕРЕОЦІНКА ФОРМИ

Гладко написаний абзац звучить переконливо — незалежно від якості доказів.
→ Форма НЕ є доказом змісту.

⚠ ВИТІК ЧУТЛИВИХ ДАНИХ

Не вставляти у комерційні API PII-дані фігурантів.
→ Для чутливих кейсів: локальні моделі (Ollama + Gemma/Llama).

★ **Аналітик, що перестає перевіряти AI-висновки, перестає бути аналітиком.**

ПЕРЕРВА

5 хвилин

Після перерви:

→ Obsidian як база знань розслідування · демо 2: живий vault і Dataview

→ Python-конвеєр: pandas → Plotly → Jinja2; карта у звіті

→ Агентні воркфлоу · демо 3: конвеєр звіту й агент через MCP

→ Резюме і домашнє завдання

Obsidian як OSINT knowledge base

Чому Obsidian для OSINT

→ Усі файли — звичайні .md (git-версіонування)
→ [[Посилання]] → автоматичний граф зв'язків
→ YAML frontmatter → Dataview-запити
→ Canvas → візуальна карта розслідування
→ Без хмари — дані залишаються локально

▸ ДЕМО 2 · граф зв’язків покажу в живому vault

Структура vault

```
Investigation-Vault/
├── 00-Дашборд.md
├── 01-Суб'єкти/
│   ├── Person-Іванов.md
│   └── Account-@ivan_ua.md
├── 02-Події/
├── 03-Докази/
├── 04-Джерела/
└── 05-Звіти/
```

Шаблони Obsidian: YAML frontmatter

Шаблон: Суб'єкт (Person)

```
---
тип: особа
назва: Іванов Іван Іванович
псевдоніми:
  - "@ivan_ua"
  - "ivan.petrenko90@gmail.com"
статус: підтверджено
впевненість: висока
теги: [фігурант, оператор-впливу]
організації:
  - "[[Organization-TechUA]]"
джерела:
  - "[[Source-TelegramChannel]]"
дата_додавання: 2026-05-26
---
## Відомі факти
## Встановлені зв'язки
## Прогалини
```

Шаблон: Подія (Event)

```
---
тип: подія
назва: Координована публікація контенту
дата: 2026-01-15
час: "14:22"
місце: Telegram / онлайн
учасники:
  - "[[Account-@channel_a]]"
  - "[[Account-@channel_b]]"
докази:
  - "[[Screenshot-001]]"
статус: підтверджено
теги: [CIB, координація, вибори]
---
```

💡 *Core-плагін Templates підставляє {{date}} і {{title}}; Templater додає логіку. Назви полів — без апострофів: інакше Dataview їх не прочитає.*

Dataview: живий дашборд розслідування

SQL-подібні запити до YAML frontmatter усіх нотаток vault у реальному часі.

Непідтверджені суб'єкти

```
TABLE назва, впевненість, теги, дата_оновлення
FROM "01-Суб'єкти"
WHERE статус = "ймовірно"
SORT дата_оновлення DESC
```

Таймлайн подій

```
TABLE назва, час, учасники, статус
FROM "02-Події"
SORT дата ASC
```

Суб'єкти без зв'язків (прогалини)

```
TABLE назва, дата_додавання
FROM "01-Суб'єкти"
WHERE !організації
SORT дата_додавання ASC
```

Дашборд (JavaScript)

```
const s = dv.pages('"01-Суб\'єкти"');
dv.paragraph(`Фігурантів: ${s.length}`);
// підтверджено: ${s.filter(p=>p.статус==="підтверджено").length}
```

▶ **ДЕМО 2 · ці запити працюють у нотатці 00-Дашборд живого vault**

Демо 2 · Obsidian як база знань

Що показую

1. Vault — це папка з .md: нотатки сутностей із frontmatter, звіти, сирі дані

2. Нова нотатка за шаблоном «Суб'єкт»: YAML заповнюється сам

3. 00-Дашборд: запити Dataview — непідтверджені суб'єкти, таймлайн, суб'єкти без зв'язків

4. Claude Code пише нотатку прямо у vault → вона одразу з'являється в таблиці

5. Graph view: сутності, події і звіт в одному графі

На що дивитись

→ Frontmatter = колонки таблиці; нотатка = рядок

→ Статус і впевненість — поля, а не слова в тексті: їх можна фільтрувати

→ «Суб'єкти без зв'язків» — готовий список прогалин

→ Дашборд не редагують руками: він перебудовується з нотаток

→ Усе локально: жодних хмар, vault можна тримати в git

Кейс А · entities/, reports/, inbox/ із Заняття 9

Кейс В · Кейс-канали/01-Суб'єкти, 02-Події

База знань — це шаблон плюс запит. Шаблон тримає структуру, запит показує прогалини.

Python pipeline: від сирих даних до звіту



Мінімальний робочий приклад

```
# 1. Дані → pandas: читання й агрегація
df = pd.read_csv("telegram_events.csv", parse_dates=["timestamp"])
daily = df.groupby("date").agg(messages=("message_id", "count"))
# 2. Plotly: графік у PNG
fig = px.line(daily, y="messages", title="Активність по днях")
fig.write_image("activity.png")
# 3. Метрики → шаблон Jinja2 → Markdown
stats = {"total": len(df), "peak_day": daily["messages"].idxmax()}
report = env.get_template("report.md.j2").render(**stats)
Path("report.md").write_text(report, encoding="utf-8")
```

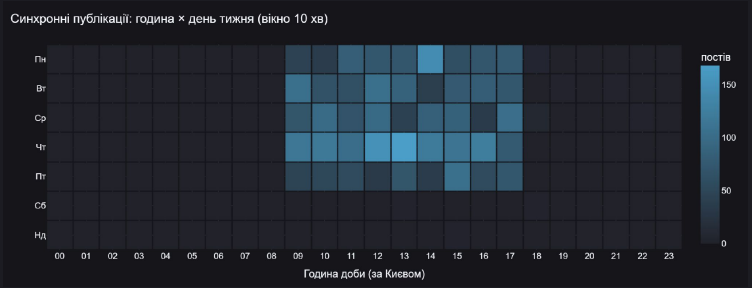
Повний скрипт на 250 рядків — make_report.py; у демо 3 запускаю його наживо.

Jinja2: шаблон звіту

```
# SPOT REPORT — {{ object }}
**Дата:** {{ now }} · **Статус:** авточернетка, не підписано
## BLUF
Зафіксовано **{{ records_unique }} санкційних записів** у **{{ jurisdictions }} юрисдикціях**; останній — {{
last_date }}.
## Динаміка
{{ img("timeline") }}
{% if recent_count > 0 %}
> ⚠ Нові записи за 36 місяців до дати збору: {{ recent_count }}
{% endif %}
## Прогалини
{% for gap in llm_gaps %}
- [ ] {{ gap }}
{% endfor %}
```

{{ }} — змінні з Python. {% if %} і {% for %} — логіка. Один шаблон → скільки завгодно звітів. Це шаблон із демо 3.

Автоматичні візуалізації: Plotly для OSINT



Heatmap: година × день тижня

Показує, коли виходять синхронні публікації
→ робочий графік оператора: будні, 09–18

```
go.Heatmap(z=pivot.values, x=hours, y=days)
```



Матриця «хто з ким»

Скільки разів пара каналів дала однаковий текст
→ кластер видно як суцільний блок

```
go.Heatmap(z=matrix.values, x=top, y=top)
```



Стовпчики за роками і джерелами

Коли і хто вносив об’єкт до списків
→ хвилі 2015 і 2022 років, записи без дат окремо

```
px.bar(by_year, x="year", y="records", color="jurisdiction")
```

Усі три графіки — реальний вихід make_report.py з демо 3. Теми light і dark, PNG для звіту, HTML для інтерактиву.

Ще в матеріалах уроку: scatter метрик мережі (betweenness × PageRank), анімований bar race.

AI-генерація коду для OSINT-скриптів

Приклад промпту для генерації коду:

Напиши Python скрипт для наступного завдання:

Вхідні дані: CSV файл telegram_messages.csv
Колонки: timestamp (ISO8601), channel_id (str), message_id (int), text (str)

Задача:
1. Знайди всі канали, що публікували ідентичний текст (cosine similarity > 0.9) у 10-хв. вікні
2. Виведи: пари каналів + кількість збігів + приклад тексту
3. Збережи результат у coincident_posts.csv

Бібліотеки: pandas, sklearn (TfidfVectorizer, cosine_similarity)

Що перевірити перед запуском:

- ☐ Правильне кодування (encoding='utf-8')
- ☐ Немає hardcoded шляхів
- ☐ Обробка порожніх значень (dropna)
- ☐ Ліміт на розмір вибірки (OOM ризик)

Ітеративний рефайнинг:

1. Отримав скрипт → запустив
2. Помилка: KeyError: 'channel_name'
3. → Claude: «Скрипт падає з KeyError.
   Ось перші 3 рядки CSV: [вставити]. Виправ.»
4. Виправлений скрипт ✓

∅ 2–4 ітерації до робочого скрипту

Карта в OSINT-звіті: коли і як

Карта додає цінності:

Подія прив'язана до конкретної локації

Показати просторовий патерн / кластер

Верифікація: «де заявлено» vs «де насправді»

Маршрут або переміщення є частиною знахідки

Карта зайва:

Немає реального просторового змісту

Точність геолокації <30 км — крапка вводить в оману

Дублює таблицю без додаткового сенсу

Стандарти публікаційної карти:

Масштабна лінійка + Північ

читач розуміє відстані та орієнтацію

Легенда для кожного шару

символ без пояснення — не доказ

Джерело підложки + дата знімку

знімки змінюються і видаляються

Координатна сітка або bbox

незалежна перевірка локації

CRS / проєкція

обов'язково при вимірах відстаней

Промпт: описати просторову знахідку словами

«Я маю координати 12 подій з атрибутами [тип, дата, джерело]. Опиши просторовий патерн для секції Findings: кластеризація, відстані, охоплений регіон. Лише факти — без оцінювальних суджень.»

QGIS: від даних до звітної карти

CSV / GeoJSON (lat, lon + attr) → Add Layer → QGIS canvas → QuickMapServices Супутник підложка → Heatmap / Buffer / Join → Print Layout Експорт PNG 300 dpi

Print Layout — обов'язкові елементи:

Add Item → Map	вставити вид карти в макет
Add Item → Scale Bar	масштабна лінійка
Add Item → North Arrow	Північ
Add Item → Legend	авто-легенда зі стилів шарів
Add Item → Label	джерело, CRS, дата
Export as Image	PNG 300 dpi → у Markdown / Word / PDF

Підготовка CSV у Python перед QGIS:

```
import pandas as pd

df_map = df[[
   "lat", "lon",
   "event_type", "date", "source"
]].dropna(subset=["lat", "lon"])

df_map.to_csv(
   "events_for_map.csv",
   index=False,
   encoding="utf-8"
)

# Обов'язково: lat/lon у десятковому
# форматі (50.4501, а не 50°27'N)
```

QGIS сьогодні без живого показу. Карту в демо 3 будує Plotly; чек-лист ліворуч чинний для будь-якого інструмента.

Альтернативи QGIS: вибір інструменту під задачу

Інструмент	Сильна сторона	Обмеження	Коли обирати
QGIS	Повний GIS-аналіз, OSM-запити, PDF-друк	Крива навчання	Аналіз, звіт для друку
Kepler.gl	Мільйони точок, Trip-анімація, HTML-звіт	Немає spatial join	Великий датасет, веб-публікація
Google Earth	Historical Imagery, KMZ/KML, безкоштовно	Закритий, не аналітичний	Швидка геолокаційна верифікація
Felt	Сучасний колаборативний веб-редактор	Хмарний — не для чутливих	Командна робота, швидка публікація
uMap	Open source, self-hostable, OSM підложки	Обмежені шари, нема аналітики	Self-hosted для чутливих кейсів
Datawrapper	Хороплет-карти для публікацій за 5 хв	Тільки агреговані дані, без точок	Choropleth по країнах і регіонах

Правило:

Чутливий кейс → QGIS / self-hosted uMap
Багато точок → Kepler.gl
Швидка перевірка → Google Earth Pro

Kepler.gl у Python:

```
map_1 = KeplerGl(height=600)
map_1.add_data(data=df_map, name='events')
map_1.save_to_html('report_map.html')
```

Агентний AI: ReAct патерн

ReAct = Reason + Act | LLM розбиває задачу на кроки → викликає інструменти → оцінює → продовжує.

1

THINK: Спочатку перевірю WHOIS для реєстрації і власника

OBSERVE: Реєстратор: Namecheap, 2025-11-03, email: admin@fastmail.com

2

THINK: Email admin@fastmail.com — перевірю витоки баз

OBSERVE: Знайдено в 2 витоках (LinkedIn 2021, Canva 2019)

3

THINK: Перевірю пасивний DNS для пов'язаних доменів

OBSERVE: IP 185.220.101.x — також хостить evil-news.net, fake-media.org

4

THINK: Достатньо для spot report. Формую висновки.

ACT: generate_report(findings) → domain_brief.md

Інструменти Claude Code

WebSearch	Пошук у відкритих джерелах
Bash	whois, curl, Python скрипти
Read/Write	Файли, нотатки, звіти
MCP Gephi	Мережевий аналіз
MCP Neo4j	Запити до БД
MCP n8n	Воркфлоу збору (Заняття 9)

Claude як OSINT-агент: практичний приклад

Задача: «Проведи OSINT-аналіз домену [target.example.com]. WHOIS → DNS → пошук → SSL → BLUF-звіт.»

$ bash("whois target.example.com")	→ Реєстратор, власник, дата реєстрації
$ bash("dig target.example.com ANY +short")	→ A, MX, TXT, NS записи
$ web_search("target.example.com pastebin")	→ Витоки, згадки у відкритих джерелах
$ bash("openssl s_client … | openssl x509")	→ SSL: CN, організація, термін дії
$ write("domain_brief.md", report_content)	→ Готовий BLUF-звіт зберігається

★ Аналітик описує ЦІЛЬ — агент адаптує план. Кожен крок задокументований і відтворюваний.

Демо 3 · Конвеєр звіту й агент

Що показую

1. make_report.py: дані → pandas → три графіки Plotly → шаблон Jinja2 → Spot Report у vault

2. Відкрити звіт в Obsidian: BLUF із цифр, графіки, блок-попередження з умови {% if %}

3. AI пише код: промпт на четвертий графік → зміна → запуск → перевірка PNG

4. Агент: ціль одним абзацом → через MCP n8n змінює об'єкт і запускає збір → міст у vault → Spot Report

5. Трейс агента: THINK · ACT · OBSERVE на кожному кроці

На що дивитись

→ pandas знаходить дублі: 32 сирі записи — це 24 унікальні рішення

→ Шаблон не вигадує суджень: рівень впевненості лишається порожнім для аналітика

→ Згенерований код читаємо і запускаємо, а не приймаємо на віру

→ Агентові дано інструменти, межі і правило безпеки: ключів він не бачить

→ Трейс — це методологія звіту: кожен крок можна відтворити

Кейс А · --case roe: санкційні записи, компанії «Ростеху», карта

Кейс В · --case channels: 17 193 повідомлення, 47 каналів

Один шаблон — скільки завгодно звітів. Аналітик описує ціль і межі, перевіряє результат і ставить підпис.

Кейс: повний OSINT-воркфлоу з AI

Розслідування координованої дезінформаційної кампанії | Без AI: 2–3 дні → З AI: ~2 години

ЗБІР 30 хв	Groupint → 47 Telegram-каналів → Neo4j Claude (агент): автоматичний WHOIS всіх доменів
МЕРЕЖА 20 хв	gephi-ai + Claude: modularity, betweenness, PageRank → автоматичний PNG + GEXF
КОНТЕНТ 25 хв	Python (Claude-generated): cosine similarity матриця → 11 каналів з ідентичним контентом ±3 хв
СИНТЕЗ 15 хв	Промпт R.O.C.E. → ACH матриця: 3 гіпотези → H1 (координована операція) найімовірніша
ЗВІТ 30 хв	Jinja2 шаблон → чернетка Claude: стиль + рівні впевненості Аналітик: атрибуція, підпис → PDF

Ручна робота аналітика: ~45 хвилин. Решта — AI і автоматизація.

РЕЗЮМЕ

Урок 16 — ключові принципи

Стандарти звіту

BLUF першою фразою · 7-секційна структура · Рівні впевненості явно · Методологія обов'язкова · Правило 3 джерел

Prompt Engineering

R.O.C.E. = роль + ціль + контекст + докази · Конкретна задача = конкретний промпт · ACH через AI

Автоматизація та візуалізація

Obsidian + Dataview · Python + Jinja2 · Plotly · QGIS Print Layout → публікаційна карта · Kepler.gl / Datawrapper для веб

Агентні воркфлоу

ReAct: Think → Act → Observe · Claude Code як OSINT-агент · Описуй ЦІЛЬ — агент адаптує план

«AI прискорює процес — аналітик несе відповідальність за висновки.
Підпис під звітом завжди людський.»

ДОМАШНЄ ЗАВДАННЯ

AI-асистований аналітичний продукт

Рівень 1 · обов'язковий

→ Spot Report українською, від 400 слів: BLUF, ключові судження, аналіз, прогалини, оцінка джерел

→ Задокументувати AI-інструмент і використані промпти

→ Obsidian + Dataview + Templater: три нотатки (два суб'єкти, одна подія)

→ Запит Dataview на статус «ймовірно», скриншот результату

Рівень 2 · розширений

→ Python-скрипт: CSV → щонайменше два агрегати і два графіки → Markdown через Jinja2

→ Записати промпти і кількість ітерацій до робочого скрипту

→ ACH-матриця: від трьох гіпотез і п'яти доказів, власна редакція результату AI

→ Бонус +10: публікаційна карта, від п'яти подій (lat, lon, event_type, date, source)

Рівень 3 · дослідницький

→ Агентний воркфлоу в Claude Code на власній OSINT-задачі

→ Зберегти трейс: кроки, інструменти, результати

→ Оцінити дії агента: де помилився, що довелося виправити

→ Фінальний Spot Report із власною редакцією

Формат здачі і критерії оцінювання — на платформі. **Підпис під звітом — ваш, а не моделі.**